Motiv

Production-Readiness & Security Audit
Full-application review ahead of public launch

6 July 2026 | audited @ ff1bdeb | v1.2 - same-day fixes (cdc7dec) + supplier SLA review

How this audit was performed

Static review of the full codebase (middleware, all 36 API route handlers, auth flows, Supabase schema/RLS, storage policies, security headers, dependencies, legal pages, PWA/service worker surface), plus automated verification runs:

TypeScript (tsc --noEmit)	PASS - no errors
ESLint (next/core-web-vitals)	PASS - no warnings or errors
Production build (next build)	PASS - exit code 0
Unit tests (vitest)	PASS - 18/18 (lib/health only)
npm audit (production deps)	2 findings - 1 high (next), 1 moderate (postcss)
API route sweep	36/36 routes checked for auth + rate limiting

Executive summary

Motiv's security architecture is genuinely strong for an application at this stage: a strict nonce-based Content-Security-Policy, consistent per-route authentication, distributed rate limiting, deny-by-default row-level security, private storage buckets with signed URLs, and a closed signup privilege-escalation path. The engineering fundamentals are in place.

At audit time three blockers stood between Motiv and a public launch. **Two were fixed the same day** (commit cdc7dec): the broken Individual-accounts workflow, and the ambiguous migration state (confirmed applied to live, folded into schema.sql). The remaining blockers are non-code: complete the legal pages (POPIA) and move to commercial hosting tiers with backups. Section 6 lays out the full path from today's state to a 9.5/10 production-grade application.

OVERALL SCORE	7.4 / 10	was 7.0 at audit - Individual workflow + migration state fixed same day; 2 non-code blockers remain

Section scores

Two columns: the score at audit time, and the score after the same-day fixes (commit cdc7dec). Section 6 gives the concrete route from each current score to 9.5+.

Section	At audit	Now	Verdict
API security & authorisation	8.5	8.5	Consistent auth, rate-limit, tenant + role checks on every route
Database & row-level security	8.0	8.5	Deny-by-default RLS; migration bookkeeping now resolved (folded + deleted)
Security headers / transport / CSP	9.5	9.5	Nonce CSP with strict-dynamic; full header suite; HSTS preload
File storage	9.0	9.0	Private buckets, signed URLs, MIME + size limits
Individual-accounts feature	5.0	8.0	403 workflow bug FIXED (cdc7dec); realtime + end-to-end test still pending
Dependencies	6.0	6.0	Next.js 14 carries published high-severity advisories - upgrade planned
Legal / POPIA compliance	4.0	4.0	Privacy + terms are placeholder templates (launch blocker)
Infrastructure & operations	6.0	6.0	Non-commercial hosting license; no DB backups (launch blocker)
Code quality & tests	8.5	8.5	Clean build/lint/types; thin test coverage outside lib/health

Scoring: 9-10 production-grade | 7-8 solid, minor gaps | 5-6 material gaps | below 5 not fit for public exposure.

1. Launch blockers found at audit

BLOCKER 1 - Individual accounts: core workflow returned 403 Forbidden - FIXED (cdc7dec, 6 Jul 2026)

`app/api/tickets/[id]/transition/route.ts | app/api/tickets/[id]/route.ts`

The ticket-transition endpoint rejected any user whose profile has no company_id - and Individual (general-public) users have none by design. The guard at `app/api/tickets/[id]/transition/route.ts` blocked Individuals before the ownership check (which already supported them) was ever reached.

The Individual ticket-detail page renders ApproveSignoffCard, CancelTicketCard, CloseOutButton, RaiseSnagButton, RequestEvidenceButton and AcceptSnagScheduleCard - every one posts to /transition, so every click failed with 'Forbidden'. The same guard also blocked Individuals from editing or deleting their own open tickets.

Resolution: Applied in commit cdc7dec: the role 'individual' now passes the tenant guard and ownership is enforced via created_by (transition, PATCH and DELETE handlers). Verified with tsc, lint and the test suite. Remaining: click-test the full Individual lifecycle on the deployed app (log job, assign supplier, approve quote, approve sign-off, close out).

BLOCKER 2 - Privacy policy & terms are unfinished templates - OPEN

`app/privacy/page.tsx | app/terms/page.tsx`

app/privacy/page.tsx and app/terms/page.tsx still contain bracketed placeholders: [DATE], [Company legal name], [reg no.], [Name], [email], [phone], [physical address], plus notes such as '[Confirm your hosting regions and safeguards]' and '[State your specific retention periods]'.

Publishing a POPIA privacy policy with placeholder text is a compliance failure, not a cosmetic issue - POPIA requires a named Information Officer and accurate processing descriptions. It also immediately signals an unfinished product to users.

Recommended fix: Complete every bracketed field (legal entity, registration number, Information Officer details, retention periods, hosting regions), set a real 'Last updated' date, and have a lawyer review before launch. The template structure itself is sound - this is content work, not code work.

BLOCKER 3 - Hosting license & zero backup story - OPEN

`docs/GO_LIVE_CHECKLIST.md | docs/INFRASTRUCTURE_TIERS.md`

Vercel Hobby's license is non-commercial. A public, commercial Motiv deployment on Hobby violates the terms of service and risks suspension - an outage caused by a ToS enforcement action, at launch, with no fallback.

Supabase free tier has no point-in-time recovery and no automated backups. Once real customer data lands (tickets, quotes, compliance certificates, invoices), a single bad migration or accidental deletion is unrecoverable.

Recommended fix: Purchase Vercel Pro and Supabase Pro (approx. $45/month combined - already costed in the project's go-live checklist) BEFORE public launch, and enable PITR/daily backups the same day. This also unblocks the hourly SLA cron and leaked-password protection from the deferred backlog.

2. High-priority findings

HIGH 1 - Next.js 14.2.35 carries published high-severity advisories - OPEN

`package.json (next 14.2.35) | npm audit --omit=dev`

npm audit reports a high-severity advisory set against the installed Next.js version, including: XSS in App Router applications using CSP nonces (GHSA-ffhc-5mcf-pf4q - directly relevant, Motiv uses nonce CSP), middleware/proxy cache poisoning (GHSA-3g8h-86w9-wvmq), React Server Component cache poisoning (GHSA-wfc6-r584-vfw7), several denial-of-service vectors (image optimizer, server components), and HTTP request smuggling in rewrites.

Fixes for most of these land in Next 15/16 only - a breaking upgrade. A public app running a version with a long published advisory list becomes a scripted-scanner target.

Recommended fix: Plan a dedicated Next 15 (or 16) upgrade PR: run the codemods, retest the nonce-CSP wiring (the theme inline script and middleware header handoff are the fragile parts), the Capacitor WebView, and the auth cookie flow. Do not bundle it with feature work.

HIGH 2 - Migration state was ambiguous - RESOLVED (cdc7dec, 6 Jul 2026)

`supabase/schema.sql | supabase/migrations/ (now empty)`

Five migration files sat in supabase/migrations/ (20260717 through 20260721 - the Individual accounts schema plus the signup role-escalation lock) while schema.sql contained no trace of the individual role - so the canonical schema file did not reflect production.

Confirmed with the owner: all five WERE applied to the live database. The risk scenario (production running the old signup trigger that honours a client-supplied role) did not apply - the escalation lock is live.

Resolution: Applied in commit cdc7dec: all five migrations folded into supabase/schema.sql (nullable company/store on tickets + six child tables, roles seed, tickets_created_by_idx, and the final handle_new_user that clamps self-signup to 'individual') and the migration files deleted, per the schema-is-canonical rule. schema.sql mirrors production again.

HIGH 3 - WhatsApp webhook accepts unsigned requests when the app secret is unset - OPEN

`app/api/webhooks/whatsapp/route.ts:409-416`

verifyWebhookSignature() returns true (fail-open) when WHATSAPP_APP_SECRET is not configured. This is documented and deliberate (the Meta App Secret arrives only when WhatsApp Business registration completes), but once the endpoint is public anyone who discovers the URL can forge webhook payloads - creating tickets, driving sessions, and triggering Groq API spend.

Recommended fix: Set WHATSAPP_APP_SECRET in Vercel the day Meta registration completes. Better: make the check fail-closed in production regardless - `if (process.env.NODE_ENV === 'production' && !WA_APP_SECRET) return false` - so a missing env var can never silently disable signature verification.

3. Medium-priority findings

MEDIUM 1 - Realtime updates likely dead on Individual pages

`supabase/schema.sql (app_can_see_ticket) | app/individual/layout.tsx`

Row-level-security helper app_can_see_ticket() requires the ticket's company_id to match the caller's - Individual tickets have company_id NULL, so the browser-side Supabase client (which RealtimeRefresh subscriptions run through) can never see them. Server-rendered pages work because they use the service-role client, but live updates on /individual pages will silently never fire.

Recommended fix: Either add a precise RLS read policy for owner-scoped tickets (`created_by = auth.uid() AND company_id IS NULL`) so realtime works, or accept refresh-on-navigation for Individuals and remove the subscription. Verify actual behaviour on /individual first.

MEDIUM 2 - Storage upload policy is any-authenticated-user, any path

`supabase/schema.sql (storage policies)`

The three bucket upload policies check only auth.role() = 'authenticated'. Any logged-in user - including free self-signup Individuals - can upload 15 MB files into any bucket under any path, without limit on count. MIME types and file size are capped and buckets are private, so the practical risk is storage-quota abuse (free tier: 1 GB) rather than data exposure.

Recommended fix: Post-launch hardening: enforce a per-user path prefix in the upload policy (e.g. name must start with auth.uid()) or move uploads behind a server route with per-user quotas. Monitor bucket growth in the admin infra dashboard meanwhile.

MEDIUM 3 - Documentation drift (three instances that will mislead future work)

`docs/PRODUCTION_READINESS.md | CLAUDE.md | supabase/schema.sql`

docs/PRODUCTION_READINESS.md still says buckets are public and rate limiting is in-memory - both since fixed (buckets are private; Upstash Redis is wired). CLAUDE.md describes the old role model ('client' role, NEXT_PUBLIC_ADMIN_EMAILS as required - that variable is no longer read anywhere in code). schema.sql references the removed /api/files/sign.

Recommended fix: Half-hour cleanup pass: update PRODUCTION_READINESS.md to current state, refresh CLAUDE.md's role/env sections, drop the stale comment in schema.sql. Stale security docs are worse than no docs - the next contributor will 'fix' solved problems or trust wrong assumptions.

MEDIUM 4 - Test coverage is thin where change velocity is highest

`lib/workflow.ts | lib/health/*.test.ts (existing pattern to copy)`

18 passing tests, all in lib/health (the scoring engine). The ticket workflow state machine - lib/workflow transition rules, role permissions per status - has zero tests despite being the highest-churn logic in recent git history (the Individual 403 blocker in this report is exactly the class of regression a transition-matrix test would have caught).

Recommended fix: Add a vitest suite for resolveTransition(): for each (status, action, role) triple assert allowed/denied, including the individual role. Pure functions, no DB - fast to write, and it turns every future workflow edit from 'hope' into 'checked'.

MEDIUM 5 - Dev-dependency staleness

`package.json devDependencies`

ESLint 8 is end-of-life (upgrade path: ESLint 9 + next lint migration). The xlsx package is pinned to a CDN tarball (0.20.3) - deliberate and currently fine, but it will not surface in npm audit, so advisories must be checked manually.

Recommended fix: Fold the ESLint 9 migration into the Next 15/16 upgrade PR (eslint-config-next moves in lockstep). Add a quarterly reminder to check SheetJS advisories manually.

4. What is genuinely strong (keep doing this)

Strict nonce-based CSP done right

Per-request nonce generated in middleware, script-src 'self' 'nonce-...' 'strict-dynamic', no unsafe-inline in production, unsafe-eval dev-only. Paired with HSTS preload (2 years, includeSubDomains), X-Frame-Options DENY, nosniff, Referrer-Policy and a Permissions-Policy. This is a stronger header posture than most production SaaS apps.

Consistent API defence-in-depth

Every route follows authenticate, rate-limit, tenant guard, role check, then act. All 36 routes verified; the only exceptions are by design (token-gated supplier onboarding, HMAC-verified webhook, CRON_SECRET-guarded crons). Rate limiting is distributed via Upstash Redis with a graceful in-memory fallback.

Signup privilege escalation closed at the database layer

The signup trigger clamps any client-supplied role to 'individual'; privileged roles are only ever set by trusted service-role paths (admin invites, token-gated supplier onboarding). The guard lives in the trigger, not the UI - the right layer. Confirmed applied to production and now reflected in the canonical schema.sql.

Private storage with signed URLs

All three buckets private, 15 MB caps, MIME allow-lists, reads via short-lived server-side signed URLs only. The earlier public-bucket exposure of compliance certificates and invoices was found and fixed properly.

POPIA data-subject rights implemented

Account deletion anonymises PII, scrambles the login email, bans the auth user, removes push subscriptions and revokes ALL sessions globally - with the residual ~1h JWT window documented rather than hidden. A data-export endpoint exists alongside it.

Deny-by-default RLS with documented triage

RLS on every table; tables without policies are deliberately service-role-only. Supabase advisor findings were triaged in writing, including which warnings are safe-by-design - so nobody 'fixes' them into a hole later.

Webhook verification hygiene

HMAC over the raw body, timing-safe comparisons for both the signature and the verify token, loud logging on rejection.

Open-redirect protection on the auth callback

The next parameter is validated against protocol-relative URLs, backslash tricks, absolute URLs and userinfo tricks before redirecting.

5. Recommended path to production

Ordered plan; steps 1-3 were completed on audit day. Remaining blocker work is legal/content plus the tier purchases.

#	Action	Size	When
1	Resolve migration state (HIGH 2): confirmed applied to live; folded into schema.sql; migration files deleted	1-2 h	**DONE 6 Jul**
2	Fix Individual 403s (BLOCKER 1): role 'individual' passes the tenant guard in transition + PATCH + DELETE; ownership via created_by	2-3 h	**DONE 6 Jul (cdc7dec)**
3	Verify the fix end-to-end on the deployed app: log job, assign supplier, approve quote, approve sign-off, close out as an Individual	1 h	Next deploy
4	Add transition-matrix tests covering all roles incl. individual (MEDIUM 4)	2-4 h	This week
5	Complete privacy + terms content, lawyer review (BLOCKER 2)	Content + legal	Before launch
6	Decide SLA priority timings (section 7: P1/P2 resolution, business-hours windows), align sla_rules + FALLBACK_SLA + /sla, lawyer review of the SLA	2 h + legal	Before launch
7	Purchase Vercel Pro + Supabase Pro; enable PITR/backups; enable leaked-password protection; add the hourly SLA cron (BLOCKER 3)	1 h + ~$45/mo	Before launch
8	Set production env vars: UPSTASH_REDIS_*, NEXT_PUBLIC_SENTRY_DSN, CRON_SECRET; WHATSAPP_APP_SECRET when Meta registration completes; make the webhook fail-closed in production (HIGH 3)	1 h	Before launch
9	Launch smoke test: signed-URL image rendering, raw public storage URL returns 403, signup creates Individual only, each role's dashboard loads, WhatsApp intake end-to-end	2 h	Launch day
10	Next.js 15/16 + ESLint 9 upgrade PR with full CSP/Capacitor/auth retest (HIGH 1)	1-2 days	Week 1-2 after launch
11	Individual realtime decision (MEDIUM 1) + storage per-user path policy (MEDIUM 2)	0.5 day	Week 1-2 after launch
12	Docs refresh: PRODUCTION_READINESS.md, CLAUDE.md, schema comment (MEDIUM 3)	0.5 h	Week 1-2 after launch

Definition of 'production ready' for Motiv

Ready = blockers 2-3 cleared, production env vars set, Individual lifecycle verified on the deployed app, and the launch smoke test green. The high-priority items (Next upgrade, webhook fail-closed) should follow within two weeks of launch.

6. Path to 9.5/10 - production grade

9.5 overall means every section at 9.0 or higher, plus independent validation (a penetration test and a backup-restore drill). The table below lists, per section, exactly what closes the gap between the current score and 9.5+. Items are ordered by impact within each section.

Section	Now	What gets it to 9.5+
API security & authorisation	8.5	1. Make the WhatsApp webhook fail-closed in production (no secret = reject, never skip). 2. Add schema validation (e.g. zod) to every write route's body - today fields are picked manually and unknown keys ignored; validation makes malformed input an explicit 400 and documents each contract. 3. Alert (Sentry) when the rate limiter falls back to in-memory, so an Upstash outage is noticed, not silent. 4. Write audit-log rows for privileged actions (provisioning, admin account ops, role changes). 5. Commission an independent penetration test before commercial scale-up.
Database & row-level security	8.5	1. Add the owner-scoped RLS read policy for Individual tickets (created_by = auth.uid() AND company_id IS NULL) so browser reads/realtime work without widening anything else. 2. Automate schema-drift detection: a CI step (or monthly ritual) that runs export_live_schema.sql against prod and diffs it with schema.sql - drift was this audit's biggest DB finding, automation stops it recurring. 3. Re-run the Supabase Security Advisor quarterly and after every migration. 4. Once on Pro: perform an actual restore drill - a backup you have never restored is a hope, not a backup.
Security headers / transport / CSP	9.5	Already at the bar - hold it. 1. Add a report-uri/report-to endpoint so CSP violations are collected (catches regressions and real attack attempts). 2. Re-verify the nonce wiring after the Next 15/16 upgrade - it is the most fragile part of the upgrade. 3. Submit the domain to the HSTS preload list (the header already opts in).
File storage	9.0	1. Enforce per-user path prefixes in the three upload policies (object name must start with auth.uid()) - kills cross-user path collisions and makes abuse attributable. 2. Add per-user upload quotas (count + bytes) via a server route or a nightly check with alerting. 3. Surface bucket growth on the admin infra dashboard with a threshold alert.
Individual-accounts feature	8.0	1. Verify the 403 fix end-to-end on the deployed app (log job, assign supplier, approve quote, sign-off, close-out). 2. Decide + implement the realtime story (MEDIUM 1's RLS policy, or drop the subscription). 3. Add the Individual role to the transition-matrix test suite so this exact regression class is caught by CI, not by users. 4. Rate-limit review for self-signup abuse: an Individual account is free to create - confirm signup + ticket-creation limits hold up against bulk abuse.
Dependencies	6.0	1. Next.js 15/16 upgrade PR (clears the entire published advisory list, including the nonce-CSP XSS that directly affects this app) with full retest of CSP, Capacitor WebView, auth cookies. 2. ESLint 9 migration in the same PR. 3. Turn on Renovate or Dependabot so version drift becomes small weekly PRs instead of a yearly cliff. 4. Add 'npm audit --omit=dev, fail on high' to CI. 5. Quarterly manual advisory check for the CDN-pinned xlsx package (invisible to npm audit).
Legal / POPIA compliance	4.0	1. Replace every bracketed placeholder in /privacy and /terms with real entity details, retention periods and hosting regions. 2. Appoint AND register the Information Officer with the Information Regulator (a POPIA obligation, not just a page edit). 3. Lawyer review + sign-off of both documents; set the real 'Last updated' date. 4. Verify operational alignment: the retention/deletion behaviour the policy promises must match what /api/account/delete actually does (it anonymises and retains operational history - say exactly that). 5. Add the policy-acceptance checkbox at signup with a stored timestamp.

Section	Now	What gets it to 9.5+
Infrastructure & operations	6.0	1. Vercel Pro + Supabase Pro (commercial license + backups/PITR) - the two purchases gate everything else. 2. Restore drill after PITR is on. 3. A staging environment (second Vercel project + Supabase branch/project) so migrations and the Next upgrade are rehearsed, not premiered. 4. Uptime monitoring with alerting on the public URL + key API routes. 5. Vercel log drain wired to alerts on auth failures and 5xx spikes. 6. Custom SMTP for Supabase auth mail (built-in mailer is rate-limited and spam-prone). 7. A one-page incident runbook: who rotates which key, how to roll back a deploy, how to restore the DB.
Code quality & tests	8.5	1. Transition-matrix test suite for lib/workflow (every status x action x role, individual included) - pure functions, no DB, biggest bang for the buck. 2. CI pipeline (GitHub Actions): tsc + lint + vitest + build on every PR; nothing merges red. 3. Integration tests for the three route handlers fixed in cdc7dec (mock Supabase, assert authZ per role). 4. A small Playwright smoke suite: each role logs in and sees its dashboard; an Individual completes the happy-path lifecycle. 5. Keep the zero-warning lint/type baseline - it is already clean; make CI enforce it.

Suggested sequencing to 9.5

Phase A - launch gate (this week): legal content + lawyer review; Pro tiers + PITR; production env vars; webhook fail-closed; Individual lifecycle verified on deploy; smoke test. Gets the app safely public at ~7.5-8.0.

Phase B - hardening (weeks 1-3 after launch): Next 15/16 + ESLint 9 upgrade; CI pipeline with tests-on-PR; transition-matrix + route authZ tests; Individual realtime policy; storage path prefixes + quotas; docs refresh. Takes most sections to 9.0+.

Phase C - validation (month 2): staging environment; restore drill; uptime + log-drain alerting; CSP reporting; Renovate; independent penetration test. Passing pen test + drill is what justifies calling the app 9.5/10 production-grade rather than self-assessing it.

7. Supplier SLA agreement & priority-timing review

7.1 The SLA agreement (drafted, template status)

A versioned Supplier Service Level Agreement now ships at **/sla** (version constant in lib/sla.ts, same [bracketed]-template pattern as /privacy and /terms - lawyer review required before launch). Suppliers accept it electronically as the final onboarding step; the platform records the accepting user, supplier, typed-name signature, SLA version and timestamp. Bumping the version re-prompts every supplier before they can receive new work. In short, the SLA commits a supplier to: the P1-P4 response/attendance/quote/resolution targets below; make-safe obligations on emergencies; binding quotes and no work before approval; photo/COC/invoice completion evidence; free rework on workmanship snags with a bounded rework window; variation orders before extra work; defined-only SLA pauses; licence, insurance and VAT compliance; POPIA handling of client data; and performance-based suspension/removal.

7.2 Contract timing matrix (mirrors the live SLA engine)

Priority	First response	Attendance	Quote due	Resolution	Internal decision
P1 Emergency	1 h	4 h	4 h	**4 h**	4 h
P2 Urgent	4 h	8 h	8 h	**24 h**	8 h
P3 Standard	24 h	48 h	48 h	5 d	48 h
P4 Low	48 h	5 d	5 d	7 d	5 d

Source: lib/health/constants.ts FALLBACK_SLA (mirrors the seeded sla_rules). Red/orange cells are the values the review below recommends changing.

7.3 Timing scrutiny - where the numbers need work and why

SLA-1 - P1 resolution (4h) is arithmetically impossible whenever a quote is involved

`lib/health/constants.ts:71 | sla_rules (P1) | /sla clause 2-4`

P1 attendance is due at hour 4 AND resolution is due at hour 4 - the supplier must arrive and have already finished, with zero time allocated to the repair itself. Worse on the quote path: the quote is due at hour 4 and the internal approval decision gets another 4h (hour 8) - the approval deadline lands 4 hours AFTER the job was already due resolved. Any P1 needing a quote breaches by construction, and the health engine scores stores and suppliers on these numbers.

Recommended fix: Split the P1 obligation the way the industry does: keep response 1h + attendance 4h, add a MAKE-SAFE obligation on first attendance (drafted as SLA clause 3), and move full resolution to 24h. Update the sla_rules row + FALLBACK_SLA together, and rely on clause 4.3 (agreed schedule becomes the deadline) for parts-dependent repairs.

SLA-2 - P2's quote cycle consumes 16 of its 24 resolution hours

`lib/health/constants.ts:72 | app/api/tickets/[id]/transition/route.ts (accept_schedule)`

P2: quote due 8h + internal decision 8h = 16h of process inside a 24h resolution window, leaving 8h for the actual repair only if both the supplier and the approver hit their deadlines exactly. Realistic only for trivial fixes; every quoted P2 job runs hot from the moment it is logged.

Recommended fix: Either widen P2 resolution to 48h, or make the engine's existing adjusted-resolution mechanism the norm: on quote approval the clock re-baselines to the agreed schedule (adjusted_resolution_due_at already exists and is set by accept_schedule - extend it to approve_quote). The SLA draft states the re-baseline rule as clause 4.3 either way.

SLA-3 - All targets are wall-clock; no business-hours calendar

`lib/health/priority.ts:62-66 | /sla clause 2`

The engine measures minutes continuously. A P3 logged Friday 17:00 has attendance due Sunday 17:00; a P4 logged before a long weekend burns most of its window while nobody works. Suppliers get breached - and rated down - for hours nobody agreed to work, which will surface as disputes the moment real suppliers sign the SLA.

Recommended fix: Contract fix now, engine fix later. The SLA draft defines P1 as 24/7, P2 on an extended day [07:00-19:00 Mon-Sat] and P3/P4 in business hours (clause 2). Decide these windows, then add business-hours awareness to the engine's due-date maths as a scheduled enhancement - until then the engine is stricter than the contract, which is the safe direction (never breaches a supplier the contract would excuse... but the dashboards over-report lateness, so do close the gap).

SLA-4 - Snag rework has no measured deadline

`/sla clause 7 | lib/health/sla.ts`

The workflow lets a supplier propose any rework date and the manager approve it, but no SLA target bounds it and the engine does not measure snag-fix timing. A supplier can park a failed job indefinitely behind a distant proposal.

Recommended fix: The SLA draft bounds rework to the original priority's attendance window from snag assignment (clause 7.1). Follow-up engine work: measure proposed-vs-window on accept_snag and flag out-of-window proposals to the approving manager.

SLA-5 - Pause reasons are narrower in the engine than in real maintenance work

`/sla clause 10 | lifecycleFields() in transition/route.ts`

The engine pauses the SLA clock only for internal decisions (quote approval, sign-off). Real jobs also stall on approved parts lead times and denied site access - today those hours count against the supplier, which makes the performance data unfair and eventually contested.

Recommended fix: The SLA draft enumerates the allowed pauses (clause 10: client decisions, agreed schedules, client-approved parts lead time, documented denied access, force majeure) and requires contemporaneous platform records. Engine follow-up: an RM/owner-approved 'awaiting parts' pause state that sets sla_paused with a reason.

Decision needed from you: confirm (or amend) the recommended targets - P1 resolution 4h -> 24h with make-safe, P2 resolution 24h -> 48h or clock re-baseline on approval, and the business-hours windows in clause 2. Once decided: update sla_rules + FALLBACK_SLA, finalise the /sla numbers, remove the [DECISION] brackets, bump SLA_VERSION, lawyer review.

Prepared by automated code audit (Claude Code). v1.0 audited commit ff1bdeb on 6 July 2026; v1.1 same day - layout fixes, statuses updated for commit cdc7dec (Individual workflow fix + migration fold), Path-to-9.5 section. v1.2 same day - supplier SLA agreement drafted at /sla and section 7 (priority-timing review) added. Scope: application code, database schema/policies, dependencies, configuration and documentation. Runtime penetration testing and live-infrastructure review were out of scope.